

Federated Learning con Ray.io

Alessia Smeralda Brunello

Matricola: 544964

Hands-On 19

Indice

1	Introduzione al problema	2
2	Stato dell'arte	3
2.1	Idea generale	3
2.2	FedAvg	3
2.3	Tecnologie utilizzate	4
3	Metodologia usata	4
3.1	Struttura del progetto	4
3.2	Modello neurale	5
3.3	Suddivisione del dataset	5
3.4	Flusso di esecuzione	5
4	Risultati sperimentali	6
4.1	Risultati ottenuti	6
4.2	Analisi dei risultati	6
4.3	Osservazioni	7
5	Conclusione e sviluppi futuri	7
5.1	Sviluppi futuri	7

1 Introduzione al problema

L'obiettivo dell'esercitazione è implementare un sistema di *Federated Learning* per la classificazione federata di immagini del dataset MNIST.

Il dataset MNIST è composto da immagini di cifre scritte a mano. Le principali caratteristiche sono:

- immagini in scala di grigi;
- dimensione di ciascuna immagine: 28×28 pixel;
- 10 classi possibili, corrispondenti alle cifre da 0 a 9;
- 60.000 immagini per il training;
- 10.000 immagini per il test.

Il problema affrontato è quindi un problema di classificazione multiclasse: dato in input il tensore associato a un'immagine, il modello deve predire la cifra corretta.

In un approccio centralizzato, tutti i dati vengono raccolti in un unico nodo e il modello viene addestrato direttamente sull'intero dataset.

Questo approccio è semplice, ma non sempre adatto a scenari reali.

Il *Federated Learning* è utile quando:

- i dati sono distribuiti su più nodi o dispositivi;
- non si vuole trasferire il dataset completo verso un server centrale;
- si vuole simulare uno scenario in cui ogni nodo possiede solo dati locali;
- si vuole addestrare un modello globale combinando i contributi di più worker.

Nel sistema implementato sono presenti:

- un **aggregatore**, che mantiene il modello globale;
- sei **worker**, ciascuno con una porzione locale del dataset;
- un meccanismo di aggregazione dei pesi basato su *FedAvg*.

2 Stato dell'arte

Il *Federated Learning* è una tecnica di apprendimento distribuito in cui il training viene svolto localmente dai client, senza spostare direttamente i dati verso un server centrale.

2.1 Idea generale

Il funzionamento generale può essere riassunto nei seguenti passi:

1. il server inizializza un modello globale;
2. il modello globale viene inviato ai client;
3. ogni client esegue training locale sui propri dati;
4. i client restituiscono al server i pesi aggiornati;
5. il server aggrega gli aggiornamenti;
6. il nuovo modello globale viene usato nel round successivo.

2.2 FedAvg

Uno degli algoritmi più utilizzati nel Federated Learning è *Federated Averaging*, abbreviato in *FedAvg*. L'idea è calcolare una media pesata dei modelli locali restituiti dai worker.

La formula usata è:

$$w = \sum_{i=1}^K \frac{n_i}{n} w_i$$

dove:

- K è il numero di worker;
- w_i rappresenta i pesi del modello locale del worker i ;
- n_i è il numero di esempi posseduti dal worker i ;
- n è il numero totale di esempi usati dai worker;
- w è il nuovo modello globale ottenuto dopo l'aggregazione.

Nel caso implementato, il dataset è stato diviso in modo bilanciato tra i sei worker. Di conseguenza, ogni worker contribuisce allo stesso modo alla media.

2.3 Tecnologie utilizzate

Per realizzare il sistema sono state utilizzate le seguenti tecnologie:

- **PyTorch**: per definire e addestrare la rete neurale;
- **Torchvision**: per scaricare e gestire il dataset MNIST;
- **Ray.io**: per simulare un ambiente distribuito con actor remoti;
- **Makefile**: per automatizzare installazione ed esecuzione.

Ray è stato usato per rappresentare worker e aggregatore come actor remoti. In questo modo è possibile simulare un sistema distribuito anche su una singola macchina.

3 Metodologia usata

La soluzione è stata progettata in modo modulare. Ogni file ha una responsabilità specifica.

3.1 Struttura del progetto

File	Funzione
<code>model.py</code>	definizione del modello neurale MLP
<code>data.py</code>	caricamento, trasformazione e suddivisione del dataset
<code>worker.py</code>	implementazione dei worker federati
<code>aggregator.py</code>	implementazione dell'aggregatore
<code>utils.py</code>	funzioni di supporto, tra cui FedAvg
<code>main.py</code>	coordinamento dell'esecuzione
<code>Makefile</code>	automazione dei comandi

Tabella 1: Organizzazione modulare del progetto.

3.2 Modello neurale

Il modello utilizzato è un *Multi-Layer Perceptron* semplice.
La sua architettura è composta da tre layer fully-connected:

- input: immagine 28×28 , trasformata in vettore da 784 elementi;
- primo layer: $784 \rightarrow 256$;
- secondo layer: $256 \rightarrow 128$;
- terzo layer: $128 \rightarrow 10$.

I primi due layer utilizzano la funzione di attivazione ReLU. L'ultimo layer produce 10 valori, uno per ogni possibile classe.

La funzione di loss utilizzata è: `CrossEntropyLoss`

Questa funzione è adatta a problemi di classificazione multiclasse.

3.3 Suddivisione del dataset

Il training set MNIST contiene 60.000 esempi. Nell'esperimento è stato suddiviso tra 6 worker:

$$60000/6 = 10000$$

Quindi, ogni worker ha ricevuto 10.000 esempi.

3.4 Flusso di esecuzione

Il funzionamento del sistema può essere riassunto nel seguente schema:

1. avvio di Ray in locale;
2. download e caricamento del dataset MNIST;
3. suddivisione del training set tra i 6 worker;
4. creazione dell'aggregatore;
5. creazione dei worker;
6. invio del modello globale ai worker;
7. training locale sui dati di ciascun worker;
8. restituzione dei pesi locali all'aggregatore;
9. aggregazione dei pesi tramite FedAvg;
10. valutazione del modello globale sul test set;
11. ripetizione del processo per 5 round.

4 Risultati sperimentali

L'esperimento è stato eseguito con 6 worker e 5 round federati. Dopo ogni round, l'aggregatore ha valutato il modello globale sul test set MNIST.

4.1 Risultati ottenuti

La seguente immagine mostra l'output prodotto dall'esecuzione del programma.

Sono riportati la distribuzione dei dati tra i worker e i valori di loss e accuracy ottenuti dopo ogni round federato.

```
-----Federated Learning MNIST con Ray.io-----

Numero di worker: 6
Round federati: 5
Epoche locali per round: 1
Device: cpu

Download/caricamento dataset MNIST ...
100.0%
100.0%
100.0%
100.0%

Distribuzione dei dati tra worker:
Worker 0: 10000 esempi
Worker 1: 10000 esempi
Worker 2: 10000 esempi
Worker 3: 10000 esempi
Worker 4: 10000 esempi
Worker 5: 10000 esempi

Inizio training federato ...
Round 01 | Train loss media worker: 0.5195 | Test loss: 0.2637 | Test accuracy: 92.71%
Round 02 | Train loss media worker: 0.2571 | Test loss: 0.1603 | Test accuracy: 95.02%
Round 03 | Train loss media worker: 0.1886 | Test loss: 0.1242 | Test accuracy: 96.33%
Round 04 | Train loss media worker: 0.1510 | Test loss: 0.1010 | Test accuracy: 96.89%
Round 05 | Train loss media worker: 0.1243 | Test loss: 0.0905 | Test accuracy: 97.38%

Training federato completato.
```

4.2 Analisi dei risultati

Dai risultati si osservano tre aspetti principali:

- la train loss media diminuisce da 0.5195 a 0.1243;
- la test loss diminuisce da 0.2637 a 0.0905;
- la test accuracy aumenta da 92.71% a 97.38%.

Questo andamento indica che il modello globale migliora progressivamente a ogni round federato. Il risultato mostra che l'aggregazione federata tramite FedAvg è riuscita a combinare correttamente gli aggiornamenti locali dei sei worker.

4.3 Osservazioni

L'esperimento conferma i seguenti punti:

- ogni worker allena il modello solo sui propri dati locali;
- l'aggregatore non accede direttamente ai dati di training;
- vengono scambiati solo i pesi del modello;
- FedAvg permette di ottenere un modello globale efficace;
- l'accuratezza cresce in modo regolare durante i round.

5 Conclusione e sviluppi futuri

L'esercitazione ha permesso di implementare un sistema di Federated Learning con un aggregatore e sei worker, utilizzando Ray.io per la simulazione distribuita e PyTorch per il training del modello.

Il lavoro mostra quindi come sia possibile addestrare un modello globale senza centralizzare direttamente i dati di training.

5.1 Sviluppi futuri

Possibili estensioni sono:

- usare una distribuzione dei dati non bilanciata tra i worker;
- aumentare il numero di worker;
- confrontare diversi valori di learning rate e batch size;
- sostituire la rete MLP con una rete convoluzionale;
- analizzare il costo di comunicazione tra worker e aggregatore;
- confrontare FedAvg con altri algoritmi di aggregazione.